

Document number: P0029R0

Date: 2015-09-21

Project: Programming Language C++, Library Evolution Working Group

Reply to: Geoff Romer <gromer@google.com>, Chandler Carruth <chandlerc@google.com>

A Unified Proposal for Composable Hashing

1. [Background](#)
2. [Design Overview](#)
3. [Comparison with Prior Proposals](#)
4. [Notes on Wording](#)
5. [Proposed Wording](#)

Background

Currently, if you want your type to be natively usable as a key type in a standard hash container, the only option C++ offers is to specialize `std::hash`. This mechanism suffers from a number of drawbacks:

- It does not support composition efficiently or conveniently: if you want to implement your `std::hash` specialization in terms of the specializations of your members, you'll have to invent your own way of combining their hash values, and the result will be inefficient because it repeats the costly step of reducing the temporary hashing state to a `size_t` (notably, this is the reason `std::hash` is not specialized for `std::pair`, `std::tuple`, or containers).
- It requires type authors to design and implement their own hash algorithms from scratch. This is a recipe for disaster: designing and implementing a hash algorithm well requires substantial expertise, and the consequences of doing it poorly are severe, since hash tables tend to be used in the most performance-sensitive parts of an application. Furthermore, with the advent of [hash flooding attacks](#), every hash table's hash algorithm is now part of the attack surface of every application that's exposed to untrusted input. In this environment, asking users to roll their own hash algorithm is nearly as reckless as asking them to roll their own cryptography.
- User specializations cannot be disabled with SFINAE, so you can't e.g. say that `hash<Wrapper<T>>` is defined only if `hash<T>` is defined.
- It is quite awkward and verbose, particularly due to the need to switch from the type's namespace to the `std` namespace.

LEWG has received two proposals for addressing this situation, [N3333](#) and [N3980](#), which share the same basic structure, but differ in many details because the respective authors were focusing on different requirements. This paper proposes a unified design which builds on both proposals, preserving their common structure and extending it to satisfy most if not all of their requirements.

Design Overview

For clarity, we will begin by presenting our design on its own terms, assuming no prior knowledge of N3333 or N3980. Comparisons with those proposals will be deferred to the next section. A sample implementation of this proposal can be found at <https://github.com/google/hashing-demo/tree/N0029R0>.

One of the primary goals of this design is to decouple the hash algorithm from the per-type hash customization logic. To do that, we must define an algorithm-independent way for types to expose themselves to the hash algorithm. Nearly all hash algorithms fundamentally operate on bytes, so bytes are what we'll use: you specify how to hash a type by specifying its *hash representation*, which is a sequence of zero or more `unsigned char` values that represents the original value.

If two values of the same type are equal, they must not have different hash representations, because otherwise they will have unequal hash values, and hash table lookups will fail when they should succeed. If unequal values have equal hash representations, then we have a hash collision. Now, collisions introduced by the *hash algorithm* are inevitable, but they are also manageable: the algorithm's job is to trade off collision frequency and predictability against run time, and the risks can be managed by an appropriate choice of hash algorithm (or even by making the hash table bigger). Collisions introduced by the *type*, on the other hand, are not controllable in those ways; the type author is essentially forcing the user to accept a certain rate of collisions (and worse, collisions that are usually predictable by an attacker). Nonetheless, in some cases it will be a necessary optimization for an object to cache its own hash value, and treat that as its hash representation. Consequently, we require only that with high probability, different values of the same type should have different hash representations. We emphasize that it should be quite rare for that probability to be less than 1.

This makes the hash representation ideal for the hash algorithm author, but not very well-suited for the type author. In fact, in most cases it won't be possible for the type author to specify their hash representation directly: if my equality comparison

is implemented in terms of the equality comparisons of other types, my hashing logic must similarly be implemented in terms of the hashing logic of those types. In most cases, the most natural way to specify my hash representation will be as the concatenation of the hash representations of some list of other values. For example, the hash representation of a `tuple<A,B,C>` should be the hash representations of A, B, and C, one after another. We will call this sequence of values the *hash expansion* of the value.

There's a risk that this concatenation will produce collisions. For example, if the hash representation of a container is just the hash representations of its elements, then `vector<string>{"a", "b", "c"}` and `vector<string>{"abc"}` have the same hash representation. To avoid this problem, we introduce another requirement: the hash representation of a value must not be a suffix of the hash representation of any other value of the same type (in other words, the type's hash representation must form a [suffix code](#)).

With that restriction in place, it's easy to specify the hash expansion of almost any value type:

- The hash expansion of a fundamental type can be just its hash representation, which is implementation-defined (usually but not always as its object representation).
- The hash expansion of a struct or tuple can be just the concatenation of its field values.
- The hash expansion of a sequence container can be just the concatenation of its elements, followed by a `size_t` representing the length of the sequence^[1].
- The hash expansion of a variant or discriminated union can be just its value, followed by the type discriminator.

More complex types can be represented using combinations of those basic operations^[2].

Thus, rather than expect each type to compute its hash value, or even its hash representation, we require it only to compute its hash expansion. The hash algorithm is then responsible for mechanically reducing the hash expansion to the hash representation, and then consuming that hash representation to produce a final hash value.

`hash_value()` and `hash_combine()`

This design is implemented as a mutual recursion between two overloaded functions: `hash_value()`, which is overloaded by hashable types, and `hash_combine()`, which is overloaded by the hash algorithm. The use of overloading avoids the tedious scoping issues associated with specializing a template in namespace `std`. Here's how a typical `hash_value()` overload might look in this framework:

```
class Foo {
    int i;
    string str;
    bool b;
    ...
    friend bool operator==(const Foo& lhs, const Foo& rhs) {
        return lhs.i == rhs.i && lhs.str == rhs.str && lhs.b == rhs.b;
    }

    friend std::hash_code hash_value(std::hash_code h, const Foo& foo) {
        return hash_combine(std::move(h), foo.i, foo.str, foo.b);
    }
};
```

`hash_value()`'s responsibility is to take the initial state of the hash algorithm (represented by `h`), and combine it with the hash expansion of `foo` to produce a new state, which it returns. To do that, it invokes `hash_combine()`, which is responsible for performing that combining operation for an arbitrary number of arguments. `hash_combine()`, in turn, will usually invoke `hash_value()` on those arguments, although it must handle `unsigned char` directly, and may choose to do so for other types as well.

Both functions are designed to be called via ADL (i.e. without namespace qualification). `hash_value()` is intended to be found in the namespace of the value to be hashed, so as with `swap()`, calls to `hash_value()` must be preceded by `using std::hash_value` in order to look up the overloads for built-in types. `hash_combine()` is intended to be found in the namespace of the hash algorithm, and hash algorithms are always library types, so no `using` declaration is necessary.

A crucial special case occurs when hashing strings, containers, and similar dynamically-sized structures: the hash representation contains an arbitrary number of values of the same type. With the API we've described so far, it would be necessary for `hash_value()` to iterate over those values, passing them to `hash_combine()` incrementally. However, a variety of critical optimizations require that we expose that entire range to the hash algorithm in a single function call (this is also much more convenient for the type author). We expect to eventually have a standard type akin to [boost::iterator_range](#), which will let us handle those cases cleanly by passing an `iterator_range` to `hash_combine()`. In the meantime, we propose a second function `hash_combine_range()` as a transitional measure, such that `hash_combine_range(h, begin, end)` behaves the way `hash_combine(h, iterator_range{begin, end})` eventually will.

The most important optimization that `hash_combine_range()` can apply is to avoid further recursion and hash the input range as a range of bytes. This requires that the input range is contiguous^[3], and that the optimization itself is correct. For example, it's not correct if the data contains internal padding, or if the data has multiple distinct representations for equal values (e.g. positive and negative zero for IEEE floating-point). More precisely, this optimization is correct if and only if the type has the property that two objects are equal if and only if their object representations ([basic.types]/p4) are equal. When that's true, the object representation is also a valid hash representation, so it can be used as such.

We propose a new trait class called `is_uniquely_represented` to represent this property. It will be false by default, but can be specialized by the type owner to assert that the type has that property.

Hash codes

As noted above, `std::hash_code` represents an intermediate state in the computation of the underlying hash algorithm. It is passed by value rather than by reference, for reasons that we will discuss in the next section. It is move-only because copying would likely be inefficient, and also because it would likely indicate a logic bug:

```
// User erroneously expects h to be passed by reference
std::hash_code& hash_value(std::hash_code& h, const Foo& foo) {
    hash_combine(h, foo.i);
    hash_combine(h, foo.str);
    return hash_combine(h, foo.b);
}
```

In the above code, only the final line has any effect on the final hash value, which obviously was not the user's intent. In general, `hash_code` values cannot be used more than once: every use produces a new `hash_code` which must be used instead, or its state will be forgotten. Making `hash_code` move-only makes this requirement explicit.

`std::hash_code` represents the default hash algorithm, which will be used by `std::hash` and anywhere else that the standard library needs to choose a hash algorithm. We expect most users to never need any algorithm other than the default, and we expect vendors to make every effort to ensure that's the case by providing a robust and high-quality default. In particular, it is *critical* that the default hash algorithm be randomized at program startup, so that its behavior is not consistent across runs. Otherwise, experience has repeatedly shown that users will rely on the exact behavior of the algorithm (e.g. for golden test outputs), and it will be all but impossible to change it when a better algorithm becomes available. For the foreseeable future, hash designers will be locked in an arms race with the authors of hash-flooding attacks, so we must plan for hash algorithms that have a finite (and probably short) operational life.

All that being said, there are important use cases that would benefit from user control of the hash algorithm. This design supports those usages: the type author need only make `hash_value()`'s first argument a template parameter:

```
template <typename H>
H hash_value(H h, const Foo& foo) {
    return hash_combine(std::move(h), foo.i, foo.str, foo.b);
}
```

This will automatically support any type that is a model of `HashCode` (which basically just requires it to have a suitable set of `hash_combine()` overloads). We propose that all `hash_value()` overloads in the standard library be templated in this way.

It is important to note that `hash_value()`'s contract is not strong enough to support use cases like fingerprinting or serialization, which require consistency across runs of the program, and even across builds. However, in principle users could extend this framework to accommodate such use cases, by overloading `hash_value()` to provide different behavior for different hash algorithms.

std::hash Integration

In our experience, users are extremely reluctant to explicitly specify a hash functor when declaring a hash table; if the default compiles, that's what they'll use, and if the default doesn't compile, they will define a specialization (even at risk of ODR violations) to make it compile. Consequently, barring a breaking change to the hash containers, any hashing mechanism that does not integrate with `std::hash` will be stillborn.

We therefore propose extending the `std::hash<T>` primary template to provide an `operator()(const T& t)` that's implemented in terms of `hash_value(h, t)`^[4], where `h` is an rvalue of type `std::hash_code`. Note that there is precedent for extending the primary template in this way: we already do it in C++14 to support hashing enums (LWG 2148). There have been some concerns raised about how that extension affected code that SFINAEs on `std::hash`, but any fix for the enum extension should be equally applicable to our proposed extension.

Under this scheme, existing `std::hash` specializations would continue to work normally, but it would no longer be necessary to create new ones, and users would have the option to remove existing specializations at their discretion.

Heterogeneous Lookup

C++14 added support for *heterogeneous lookup* in associative containers, which allows users to look up entries in the container using a type that doesn't match the container key type (e.g. looking up a plain pointer in a set of `unique_ptr`s). This is enabled by comparators that can compare elements of the different types. To support similar functionality in hash containers, users will need to be able to extend the hash representation guarantee across types, and guarantee that equal values of two different types will have the same hash representation. This paper does not propose full support for heterogeneous lookup, but it does try to provide the functionality that a heterogeneous lookup proposal will need (see the following section).

Comparison with Prior Proposals

By-value vs. By-reference

Perhaps the most fundamental difference between N3333 and N3980 is that in N3333, the intermediate hash state is passed by value, whereas in N3980 it is passed by reference and updated in place. These choices depend on subtle performance tradeoffs: on the one hand passing by reference may save the cost of some copy/move operations, but on the other hand optimizers generally find it much easier to deal with values on the stack (i.e. values that can be reduced to SSA variables rather than memory locations). Optimizer-friendliness matters because modern hash algorithms often have a drastically simpler "fast path" for small inputs, and pruning out the "slow path" code for known-small inputs can not only reduce code size, but also enable further optimizations.

The deciding factor for us is that pass-by-value subsumes pass-by-reference: if pass-by-reference is found to be the most efficient approach, it can be implemented in this framework by making `std::hash_code` a pointer to some state on the stack of the outermost `hash_value()` caller. By contrast, if pass-by-value is found to be the most efficient approach, we see no way to emulate it in a pass-by-reference framework. Our design appears to maximize implementation flexibility.

In practice, we expect most implementations will be a hybrid, with `std::hash_code` consisting of a small amount of state that's relevant to the optimizer, and a pointer to everything else. For example, in the [prototype implementation of farmhash](#) in this framework, the movable `HashCode` consists of a `bool` and two pointers, one of which points to roughly 128 bytes of buffer and mixing state.

Multiple Hash Algorithms

The other fundamental difference between N3333 and N3980 is that N3333 permits only a single hash algorithm, whereas N3980 makes the algorithm a template parameter, explicitly providing for user choice of the hash algorithm. As a middle ground, we propose a framework that permits multiple hash algorithms, but defines a very clear default, so that users never have to explicitly choose a hash algorithm unless they want to. We are concerned that the ability to customize the hash algorithm will prove to be a foot-gun in many cases, but there are legitimate uses for it, so we follow C++ tradition by giving users the choice and hoping they choose well.

Single vs. Multiple Initialization

N3980 has the property that the hash algorithm is initialized exactly once, and then incrementally processes the entire hash representation in order. N3333 lacks that property: every `hash_combine()` call initializes a new hash state, which may subsequently be mixed into another ongoing computation. The performance consequences of this are debatable, but the semantic consequences are not: without unbounded amounts of buffering, N3333 cannot behave as if the hash algorithm were processing the hash representation in order.

This could matter for supporting use cases like heterogeneous lookup, where we need to guarantee that structurally very different types nonetheless produce identical hash values. In those cases it's necessary that the hash value have a clear logical structure, and not "leak" implementation details like the recursion structure of `hash_combine()` calls. Consequently, this proposal follows N3980 in ensuring that the hash algorithm processes the entire hash representation in order.

Hash Algorithm API

N3333 gives the hash algorithm a broad, high-level API, consisting of both `hash_combine()` and `hash_combine_range()`. Both take as input an arbitrary number of objects of arbitrary hashable types. By contrast, N3980's API for hash algorithms consists of a single `operator()` that takes raw bytes, specified as a `void*` and a length. In effect, this provides a counterpart for `hash_combine_range(hash_code, unsigned char*, unsigned char*)`, but N3980 provides no counterpart for `hash_combine()` or any of the other forms of `hash_combine_range()`. It does describe a variadic form of `hash_append` as an optional extension, which would provide a counterpart for `hash_combine()` from the user's perspective. However, it seems to be intended as a third-party utility, not an extension point that the hash algorithm can overload.

N3980's approach has the virtue of simplicity, and makes it easy to define new hash algorithms, but the narrowness of that API gives the algorithm author no flexibility to do things like implement the optimizations for contiguous-layout types described earlier; if anywhere, they must be implemented by the type authors, in their `hash_append()` overloads. This seems like the wrong tradeoff: optimization and the attendant code complexity should go in the hash algorithm, because hash algorithms will be written rarely, and by experts, whereas `hash_append()` overloads will be written frequently, and by everybody. Furthermore the hash algorithm, unlike the hashable type, is in a position to know which optimizations are appropriate to apply. For example, for heterogeneous lookup we have to ensure that different types use the same hash representation for the same value, *and* we must ensure that we actually use the hash representation for hashing. This means we cannot apply the contiguous-layout optimization in this case, because we can't use the value representation instead of the hash representation. If the type itself is responsible for these optimizations, then it has no choice but to avoid that optimization unconditionally, penalizing use cases that have nothing to do with heterogeneous lookup. On the other hand if the algorithm is responsible for optimization, we can have a separate algorithm for heterogeneous lookup, and continue to get that optimization in ordinary use cases.

The variadic form of `hash_combine()` is also important for optimization. Exposing all the function arguments in a single scope provides valuable optimization opportunities to both the optimizer and the author of `hash_combine()`.

For all of these reasons, we have adopted N3333's approach of giving the hash algorithm a broad, high-level API.

Contiguous-layout optimizations

All three proposals have some mechanism for directly hashing the underlying bytes of an object. In N3333, this takes the form of a `hash_as_bytes()` function, which is ill-formed unless its argument is trivially-copyable, standard-layout, and has no internal padding (as indicated by a proposed `is_contiguous_layout` trait). However, this has numerous drawbacks. It is too narrow, because it has no support for arrays, and excludes many user-defined types that could usefully benefit from this optimization. At the same time it is too broad, because it accepts argument types like IEEE `float` and `double`, for which this optimization is not safe. It is also awkward to use, since it requires the caller to implement the static conditional logic necessary to call either `hash_as_bytes()` or `hash_value()`, depending on the result of `is_contiguous_layout`. Finally, it places responsibility for optimization in the wrong place: as discussed earlier, these optimizations should be implementation details of the hash algorithm, not the responsibility of user-level code.

N3980 corrected most of these problems by introducing an alternative trait, `is_contiguously_hashable`, and specializing `hash_append()` for all types where that trait is true. Unlike `is_contiguous_layout`, `is_contiguously_hashable` must be explicitly specialized for individual types, which ensures that it's neither too broad nor too narrow (barring user error).

Our proposed `is_uniquely_represented` is identical to N3980's `is_contiguously_hashable`. The new name makes the underlying generality of the concept more explicit, and draws on existing literature (*Elements of Programming* by Stepanov and McJones) rather than coining a new term. In any event, it was LEWG's preferred bikeshed color as of Rapperswil.

There may still be a place for N3333's `is_contiguous_layout`, as an implementation detail of user specializations of `is_uniquely_represented`. Without it, users are forced to do `sizeof` arithmetic on all their members to determine if their types contain internal padding, which is inconvenient and error-prone. We do not propose that here, in order to keep this a "pure library" proposal (`is_contiguous_layout` requires compiler support), but we are open to pursuing it in a follow-up proposal.

Notes on Wording

The proposed wording below provides `hash_value` overloads for every type in the standard library that has a `std::hash` specialization (as well as providing overloads for many new types). In principle we could remove the existing `std::hash` specializations, and let their functionality be provided by the primary template. However, this would require the primary template to be provided by every header that currently contains a `std::hash` specialization. This seems excessive, so we've left the specializations in place, with the exception of the specializations for fundamental types, where this issue does not arise.

We've introduced a new header `<hash>`, because no existing header seemed suitable. We've specified that `std::hash` is available via `<hash>` (in addition to `<functional>`) because the alternative would be an endless source of user confusion.

We have specified the exact hash expansions of `unique_ptr` and `shared_ptr`, in order to match their existing `hash` specializations. However, we have not specified the exact hash expansions of `pair`, `tuple`, or the containers, nor constrained them to be equivalent to each other. It is likely that a future proposal for heterogeneous lookup support will need to do this, but we prefer not to prejudge the issue.

We have tentatively omitted `noexcept` specifications from our proposal, in order to support potentially-throwing use cases without resorting to elaborate conditional `noexcept` expressions. However, a plausible case could be made for blanket use of `noexcept`, or even for complex conditionals, since the overwhelmingly common use cases should not be throwing, and may benefit from exposing that fact at compile time. We are especially open to feedback on this aspect of the proposal.

Proposed Wording

Changes are relative to N4527.

Primary wording

Add a new subclause, following [hash.requirements]:

17.6.3.X Requirements for Composable Hashing [composable.hashing.requirements]

A hash representation of a type T is a mapping R from values of type T to sequences of zero or more unsigned char values that has the following properties (where t_1 and t_2 are rvalues of type (possibly cv-qualified) T):

- If $t_1 == t_2$ then $R(t_1) = R(t_2)$.
- If $!(t_1 == t_2)$, then with high probability, $R(t_1) \neq R(t_2)$. [Note: implementations are strongly encouraged to make this probability 1 wherever possible; "with high probability" is intended to accommodate unusual types such as those that hold a cached hash value. — end note]
- $R(t_1)$ must not be a suffix of $R(t_2)$.

A type H meets the requirements of `HashCode` if it meets the requirements of `MoveConstructible`, `MoveAssignable`, and `Destructible`, and if the expressions shown in Table X are valid and have the indicated semantics. In Table X, hr denotes an rvalue of type H , hl denotes an lvalue of type H , $[i, j)$ denotes a valid range whose element type is Hashable by H , and $args$ denotes a sequence of zero or more rvalues of types that are Hashable by H . An object h of type H must behave as if it owns a sequence of zero or more unsigned char values, referred to below as the "internal state" of h .

Table X: HashCode requirements

Expression	Return type	Operational semantics
$H\ hl(hr)$ $H\ hl = hr$		post: the internal state of hl is the same as the internal state of hr before the initialization.
$hl = hr$	$H\&$	post: the internal state of hl is the same as the internal state of hr before the assignment.
$hash_combine(hr, args)$	H	Returns a value of type H whose internal state consists of the internal state of hr , followed by a hash representation of each argument in $args$, in order.
$hash_combine_range(hr, [i, j))$	H	Returns a value of type H whose internal state consists of the internal state of hr , followed by a hash representation of each element of $[i, j)$, in order.

Let t be an rvalue of type (possibly cv-qualified) T , H be a type that meets the requirements of `HashCode`, and h be an rvalue of type (possibly cv-qualified) H . A type T is Hashable by H if the expression `hash_value(h, t)` is well-formed when treated as an unevaluated operand, and returns a value of type H . The returned value's internal state must consist of the internal state of h , followed by a hash representation of t .

For the duration of program execution, a given overload of `hash_combine`, `hash_combine_range`, or `hash_value` must always use the same hash representation for a given type.

Remove the following from the <functional> synopsis in [function.objects]:

```
//Hash function specializations
template<...> struct hash<bool>;
template<...> struct hash<char>;
template<...> struct hash<signed char>;
template<...> struct hash<unsigned char>;
template<...> struct hash<char16_t>;
template<...> struct hash<char32_t>;
template<...> struct hash<wchar_t>;
template<...> struct hash<short>;
template<...> struct hash<unsigned short>;
template<...> struct hash<int>;
template<...> struct hash<unsigned int>;
template<...> struct hash<long>;
template<...> struct hash<long long>;
template<...> struct hash<unsigned long>;
```

```
template<> struct hash<unsigned long long>;

template<> struct hash<float>;
template<> struct hash<double>;
template<> struct hash<long double>;

template<class T> struct hash<T*>;
```

Revise [unord.hash] as follows:

The unordered associative containers defined in 23.5 use specializations of the class template `hash` as the default hash function. For all object types `Key` for which there exists a specialization `hash<Key>`, or for which `Key` is Hashable (17.6.3.X) by `hash_code` (20.X.2) **and for all enumeration types (7.2) `Key`**, the instantiation `hash<Key>` shall:

- satisfy the Hash requirements (17.6.3.4), with `Key` as the function call argument type, the DefaultConstructible requirements (Table 19), the CopyAssignable requirements (Table 23),
- be swappable (17.6.3.2) for lvalues
- provide two nested types `result_type` and `argument_type` which shall be synonyms for `size_t` and `Key`, respectively,
- satisfy the requirement that if `k1 == k2` is true, `h(k1) == h(k2)` is also true, where `h` is an object of type `hash<Key>` and `k1` and `k2` are objects of type `Key`;
- satisfy the requirement that the expression `h(k)`, where `h` is an object of type `hash<Key>` and `k` is an object of type `Key`, shall not throw an exception unless `hash<Key>` is a user-defined specialization that depends on at least one user-defined type.

In addition to being available via inclusion of the `<functional>` header, the `hash` template shall be available when the header `<hash>` is included.

```
template<> struct hash<bool>;
template<> struct hash<char>;
template<> struct hash<signed char>;
template<> struct hash<unsigned char>;
template<> struct hash<char16_t>;
template<> struct hash<char32_t>;
template<> struct hash<wchar_t>;
template<> struct hash<short>;
template<> struct hash<unsigned short>;
template<> struct hash<int>;
template<> struct hash<unsigned int>;
template<> struct hash<long>;
template<> struct hash<unsigned long>;
template<> struct hash<long long>;
template<> struct hash<unsigned long long>;
template<> struct hash<float>;
template<> struct hash<double>;
template<> struct hash<long double>;
template<class T> struct hash<T*>;
```

The template specializations shall meet the requirements of class template `hash` (20.9.13).

Add a new subclause, following [type.index]:

20.X Hashing support [hashing]

20.X.1 Header `<hash>` synopsis [hash.header]

```
namespace std {
    template <class T> struct hash;

    // 20.X.2, hash_code
    class hash_code;

    // 20.X.3, hash_value for core language types
    template <class H, class T> H hash_value(H h, T val);
    template <class H, class T, size_t N> H hash_value(H h, T (&arr)[N]);

    // 20.X.4, is_uniquely_represented
    template <class T> struct is_uniquely_represented;
```

```
}
```

20.X.2 Class `hash_code` [`hash.code`]

```
namespace std {  
    class hash_code {  
    public:  
        // 20.X.2.1 move operations  
        hash_code(hash_code&& h);  
        hash_code& operator=(hash_code&& h);  
  
        // disable copy from lvalue  
        hash_code(const hash_code&) = delete;  
        hash_code& operator=(const hash_code&) = delete;  
    };  
  
    // 20.X.2.2 hash combining functions  
    template <class... Ts>  
        hash_code hash_combine(hash_code h, const Ts...& ts);  
    template <class InputIterator>  
        hash_code hash_combine_range(  
            hash_code h, InputIterator first, InputIterator last);  
}
```

`hash_code` is the default model of `HashCode`. Types will be usable with `std::hash` if they are Hashable by `hash_code`.

`hash_code` is specified as if it owns a sequence of unsigned `char` values, which is referred to as its "internal state". It is unspecified how this state is initialized or accessed.

`hash_code` shall satisfy the requirements of `HashCode` (17.6.3.X).

20.X.2.1 move operations [`hash.code.move`]

```
hash_code(hash_code&& h);
```

Effects: Constructs a `hash_code` whose internal state is equal to the internal state of `h` before the construction.

```
hash_code& operator=(hash_code&& h);
```

Effects: Sets the internal state of `*this` to be equal to the internal state of `h` before the assignment.

Returns: `*this`.

20.X.2.2 hash combining functions [`hash.code.combine`]

```
template <class... Ts>  
hash_code hash_combine(hash_code h, const Ts...& ts);
```

Requires: Every type in `Ts...` is Hashable by `hash_code`.

Returns: A `hash_code` whose internal state consists of the internal state of `h`, followed by the hash representations (17.6.3.X) of the arguments in `ts`, in order.

```
template <class InputIterator>  
hash_code hash_combine_range(  
    hash_code h, InputIterator first, InputIterator last);
```

Requires: `InputIterator` is an input iterator whose value type is Hashable by `hash_code`, and `[first, last)` is a valid range.

Returns: A `hash_code` whose internal state consists of the internal state of `h`, followed by the hash representations of the elements of `[first, last)`, in order.

20.X.3 `hash_value` for core language types [`hash.value`]

```
template <class H, class T>  
H hash_value(H h, T val);
```

Requires: `H` meets the requirements of `HashCode`

Remarks: This function shall not participate in overload resolution unless `T` is an integral, enumeration, floating-point, or pointer type.

Returns: An object of type `H` whose internal state consists of the internal state of `h`, followed by a hash representation of `val`.

```
template <class H, class T, size_t N>
```



```
H hash_value(H h, T (&arr)[N]);
```

Requires: `H` meets the requirements of `HashCode`.

Remarks: This function shall not participate in overload resolution unless `T` is `Hashable` by `H`.

Effects: Equivalent to `hash_combine_range(std::move(h), begin(arr), end(arr))`.

20.X.4 is uniquely represented [hash.uniquely.represented]

```
template <class T> struct is_uniquely_represented;
```

`is_uniquely_represented` shall meet the requirements of `UnaryTypeTrait` (20.10.1), with a `BaseCharacteristic` of either `true_type` or `false_type`. The `BaseCharacteristic` shall be `true_type` only if `T` is *uniquely represented*, meaning that for any values `t1` and `t2` of type `T`, `t1 == t2` if and only if `t1` and `t2` have equal object representations (3.9).

[Note: If a type is uniquely represented, then its object representation is also a hash representation. — end note]

A program may specialize `is_uniquely_represented` for user-defined types that are not enumerated types. Such specializations must meet the requirements of the primary template.

hash_value overloads for standard library types

initializer_list

Insert the following in the `<initializer_list>` synopsis in [support.initlist]:

```
...
// 18.9.3 initializer list range access
template<class E> constexpr const E* begin(initializer_list<E> il) noexcept;
template<class E> constexpr const E* end(initializer_list<E> il) noexcept;

// 18.9.X hash support
template<class H, class E> H hash_value(H h, initializer_list<E> il);
}
```

Add a new section following [support.initlist.access]:

18.9.X Initializer list hash support [support.initlist.hash]

```
template<class H, class E> H hash_value(H h, initializer_list<E> il);
```

Requires: `H` meets the requirements of `HashCode`.

Remarks: This function shall not participate in overload resolution unless `E` is `Hashable` by `H` (17.6.3.X).

Returns: An object of type `H` whose internal state consists of the internal state of `h`, followed by a hash representation (17.6.3.X) of `il`.

error_code

Revise the synopsis of `<system_error>` in [syserr] as follows:

```
...
// 19.5.5 Hash support
template <class T> struct hash;
template <> struct hash<error_code>;

template <class H> H hash_value(H h, const error_code& e);
}
```

Revise [syserr.hash] as follows:

19.5.5 System error hash support [syserr.hash]

```
template <> struct hash<error_code>;
```

The template specialization shall meet the requirements of class template hash (20.9.13).

```
template <class H> H hash_value(H h, const error_code& e);
```

Requires: H meets the requirements of `HashCode`.

Returns: An object of type H whose internal state consists of the internal state of h, followed by a hash representation (17.6.3.X) of e.

pair

Revise the synopsis of <utility> in [utility] as follows:

```
...
// 20.3.5, pair piecewise construction
struct piecewise_construct_t { };
constexpr piecewise_construct_t piecewise_construct{};
template <class... Types> class tuple; // defined in <tuple>

// 20.3.X, pair hash support
template <class H, class T1, class T2>
H hash_value(H h, const pair<T1, T2>& p);

// 20.5, Compile-time integer sequences
template<class T, T...> struct integer_sequence;
...
```

Insert a new subclause, following [pair.piecewise]:

20.3.X Hash support [pair.hash]

```
template <class H, class T1, class T2>
H hash_value(H h, const pair<T1, T2>& p);
```

Requires: H meets the requirements of `HashCode`.

Remarks: This function shall not participate in overload resolution unless T1 and T2 are Hashable by H.

Returns: An object of type H whose internal state consists of the internal state of h, followed by a hash representation (17.6.3.X) of p.

tuple

Revise the synopsis of <tuple> in [tuple.general] as follows:

```
...
// 20.4.2.9, specialized algorithms:
template <class... Types>
void swap(tuple<Types...>& x, tuple<Types...>& y) noexcept(see below);

// 20.4.2.X, hash support
template <class H, class... Types> H hash_value(H h, const tuple<Types...>& t);
}
```

Insert a new subclause following [tuple.special]:

20.4.2.X Tuple hash support [tuple.hash]

```
template <class H, class... Types> H hash_value(H h, const tuple<Types...>& t);
```

Requires: H meets the requirements of `HashCode`.

Remarks: This function shall not participate in overload resolution unless every type in Types is Hashable by H.

Returns: An object of type H whose internal state consists of the internal state of h, followed by a hash

representation (17.6.3.X) of t.

bitset

Revise the synopsis of class `bitset` in [template.bitset]/p1 as follows:

```
...
// 20.6.3 hash support
template <class T> struct hash;
template <size_t N> struct hash<bitset<N> >;

template <class H, size_t N> H hash_value(H h, const bitset<N>&);
}
```

Revise [bitset.hash] as follows:

20.6.3 bitset hash support [bitset.hash]

```
template <size_t N> struct hash<bitset<N> >;
```

The template specialization shall meet the requirements of class template `hash` (20.9.13).

```
template <class H, size_t N> H hash_value(H h, const bitset<N>& x);
```

Requires: `H` meets the requirements of `HashCode`.

Returns: An object of type `H` whose internal state consists of the internal state of `h`, followed by a hash representation (17.6.3.X) of `x`.

Smart pointers

Revise the synopsis of `<memory>` in [memory.syn] as follows:

```
...
// 20.8.2.7 hash support
template <class T> struct hash;
template <class T, class D> struct hash<unique_ptr<T, D> >;
template <class T> struct hash<shared_ptr<T> >;

template <class H, class T, class D>
  H hash_value(H h, const unique_ptr<T, D>& x);
template <class H, class T>
  H hash_value(H h, const shared_ptr<T>& x);
}
```

Add the following to the end of [util.smartptr.hash]:

```
template <class H, class T, class D>
  H hash_value(H h, const unique_ptr<T, D>& x);

Requires: H meets the requirements of HashCode.
Effects: Equivalent to hash_combine(std::move(h), x.get()).

template <class H, class T>
  H hash_value(H h, const shared_ptr<T>& x);

Requires: H meets the requirements of HashCode.
Effects: Equivalent to hash_combine(std::move(h), x.get()).
```

chrono types

Revise the synopsis of `<chrono>` in [time.syn] as follows:

```
...
```

```
// 20.12.5.7, duration_cast
template <class ToDuration, class Rep, class Period>
constexpr ToDuration duration_cast(const duration<Rep, Period>& d);
```

```
// 20.12.5.X, duration hash support
template <class H, class Rep, class Period>
H hash_value(H h, const duration<Rep, Period>& d);
```

```
// convenience typedefs
```

```
...
```

```
// 20.12.6.7, time_point_cast
template <class ToDuration, class Clock, class Duration>
constexpr time_point<Clock, ToDuration>
time_point_cast(const time_point<Clock, Duration>& t);
```

```
// 20.12.6.X, time point hash support
template <class H, class Clock, class Duration>
H hash_value(H h, const time_point<Clock, Duration>& t);
```

```
// 20.12.7, clocks
```

```
...
```

Insert a new subclause following [time.duration.cast], and renumber following subclauses:

20.12.5.X duration hash support [time.duration.hash]

```
template <class H, class Rep, class Period>
H hash_value(H h, const duration<Rep, Period>& d);
```

Requires: H meets the requirements of `HashCode`.

Remarks: This function shall not participate in overload resolution unless Rep is Hashable by H.

Returns: An object of type H whose internal state consists of the internal state of h, followed by a hash representation (17.6.3.X) of d.

Insert a new subclause following [time.point.cast]:

20.12.6.X time point hash support [time.point.hash]

```
template <class H, class Clock, class Duration>
H hash_value(H h, const time_point<Clock, Duration>& t);
```

Requires: H meets the requirements of `HashCode`.

Remarks: This function shall not participate in overload resolution unless Duration is Hashable by H.

Returns: An object of type H whose internal state consists of the internal state of h, followed by a hash representation (17.6.3.X) of t.

type_index

Revise [type.index.synopsis] as follows:

20.14.1 Header <typeindex> synopsis [type.index.synopsis]

```
namespace std {
    class type_index;
    template <class T> struct hash;
    template<> struct hash<type_index>;

    template <class H> H hash_value(H h, const type_index& x);
}
```

Add the following to the end of [type.index.hash]:

```
template <class H> H hash_value(H h, const type_index& x);
```

Requires: H meets the requirements of `HashCode`.

Returns: An object of type H whose internal state consists of the internal state of h , followed by a hash representation (17.6.3.X) of x .

basic_string

Revise `[char.traits.require]/p1` as follows:

In [this subclause Table 62](#), x denotes a Traits class defining types and functions for the character container type `CharT`; c and d denote values of type `CharT`; p and q denote values of type `const CharT*`; s denotes a value of type `CharT*`; n , i and j denote values of type `std::size_t`; e and f denote values of type `X::int_type`; pos denotes a value of type `X::pos_type`; $state$ denotes a value of type `X::state_type`; **and** r denotes an lvalue of type `CharT`, **H denotes a type that meets the requirements of `HashCode`, and h denotes an rvalue of type H** . Operations on Traits shall not throw exceptions.

Insert the following after Table 62 in `[char.traits.require]`, and renumber following paragraphs:

In addition, if the expression `X::hash(h, c)` is well-formed when treated as an unevaluated operand, it shall have constant complexity, and shall evaluate to a value of type H , whose internal state consists of the internal state of h , followed by a sequence of zero or more unsigned `char` values which meets the following requirements. Let C denote the sequence for `X::hash(h, c)`, and D denote the sequence for `X::hash(h, d)`. Then,

- if `X::eq(c, d)`, then $C = D$,
- if `!X::eq(c, d)`, then with high probability, $C \neq D$, and
- C is not a suffix of D .

[Note: this parallels the requirements for `hash_value`, but using `X::eq()` in place of the `==` operator. — end note]

Revise the synopses of `char_traits<char>`, `char_traits<char16_t>`, `char_traits<char32_t>`, and `char_traits<wchar_t>` (in `[char.traits.specializations.char]`, `[char.traits.specializations.char16_t]`, `[char.traits.specializations.char32_t]`, and `[char.traits.specializations.wchar_t]`, respectively), applying this change to each:

```
...
static int compare(const char_type* s1, const char_type* s2, size_t n);
static size_t length(const char_type* s);
static const char_type* find(const char_type* s, size_t n,
                             const char_type& a);
template <class H> static H hash(H h, char_type c);
static char_type* move(char_type* s1, const char_type* s2, size_t n);
static char_type* copy(char_type* s1, const char_type* s2, size_t n);
static char_type* assign(char_type* s, size_t n, char_type a);
...
```

Add the following to the end of `[char.traits.specializations.char]`, `[char.traits.specializations.char16_t]`, `[char.traits.specializations.char32_t]`, and `[char.traits.specializations.wchar_t]`:

```
template <class H> static H hash(H h, char_type c);

Effects: Equivalent to hash_value(std::move(h), c).
```

Revise the synopsis of `<string>` in `[string.classes]` as follows:

```
...
// 21.6, hash support:
template <class T> struct hash;
template <> struct hash<string>;
template <> struct hash<u16string>;
template <> struct hash<u32string>;
template <> struct hash<wstring>;
```

```

template <class H, class charT, class traits, class Allocator>
H hash_value(H h, const basic_string<charT,traits,Allocator>&);

inline namespace literals {
...

```

Add the following to the end of [basic.string.hash]:

```

template <class H, class charT, class traits, class Allocator>
H hash_value(H h, const basic_string<charT,traits,Allocator>& s);

Requires: H meets the requirements of HashCode.

Remarks: This function shall not participate in overload resolution unless traits::hash(std::move(h),
charT{}) is well-formed when treated as an unevaluated operand.

Returns: An object of type H whose internal state consists of the internal state of h, followed by a hash
representation (17.6.3.X) of s.

```

Sequence containers

Revise the synopsis of <array> in [sequences.general] as follows:

```

...
template <class T, size_t N>
void swap(array<T, N>& x, array<T, N>& y) noexcept(noexcept(x.swap(y)));
template <class H, class T, size_t N>
H hash_value(H h, const array<T, N>& x);

template <class T> class tuple_size;
...

```

Revise the synopsis of <deque> in [sequences.general] as follows:

```

...
template <class T, class Allocator>
void swap(deque<T, Allocator>& x, deque<T, Allocator>& y)
noexcept(noexcept(x.swap(y)));
template <class H, class T, class Allocator>
H hash_value(H h, const deque<T, Allocator>&);
}

```

Revise the synopsis of <forward_list> in [sequences.general] as follows:

```

...
template <class T, class Allocator>
void swap(forward_list<T, Allocator>& x, forward_list<T, Allocator>& y)
noexcept(noexcept(x.swap(y)));
template <class H, class T, class Allocator>
H hash_value(H h, const forward_list<T, Allocator>& x);
}

```

Revise the synopsis of <list> in [sequences.general] as follows:

```

...
template <class T, class Allocator>
void swap(list<T, Allocator>& x, list<T, Allocator>& y)
noexcept(noexcept(x.swap(y)));
template <class H, class T, class Allocator>
H hash_value(H h, const list<T, Allocator>& x);
}

```

Revise the synopsis of <vector> in [sequences.general] as follows:

```

...

```



```

template <class T, class Allocator>
void swap(vector<T, Allocator>& x, vector<T, Allocator>& y)
    noexcept(noexcept(x.swap(y)));
template <class H, class T, class Allocator>
H hash_value(H h, const vector<T, Allocator>& x);

template <class Allocator> class vector<bool, Allocator>;

// vector<bool> hash support
template <class T> struct hash;
template <class Allocator> struct hash<vector<bool, Allocator> >;
}

```

Add the following to the end of [array.special]:

```

template <class H, class T, size_t N>
H hash_value(H h, const array<T, N>& x);

Requires: H meets the requirements of HashCode.
Remarks: This function shall not participate in overload resolution unless T is Hashable by H.
Returns: An object of type H whose internal state consists of the internal state of h, followed by a hash representation (17.6.3.X) of x.

```

Add the following to the end of [deque.special]:

```

template <class H, class T, class Allocator>
H hash_value(H h, const deque<T, Allocator>& x);

Requires: H meets the requirements of HashCode.
Remarks: This function shall not participate in overload resolution unless T is Hashable by H.
Returns: An object of type H whose internal state consists of the internal state of h, followed by a hash representation (17.6.3.X) of x.

```

Add the following to the end of [forwardlist.spec]:

```

template <class H, class T, class Allocator>
H hash_value(H h, const forward_list<T, Allocator>& x);

Requires: H meets the requirements of HashCode.
Remarks: This function shall not participate in overload resolution unless T is Hashable by H.
Returns: An object of type H whose internal state consists of the internal state of h, followed by a hash representation (17.6.3.X) of x.

```

Add the following to the end of [list.special]:

```

template <class H, class T, class Allocator>
H hash_value(H h, const list<T, Allocator>& x);

Requires: H meets the requirements of HashCode.
Remarks: This function shall not participate in overload resolution unless T is Hashable by H.
Returns: An object of type H whose internal state consists of the internal state of h, followed by a hash representation (17.6.3.X) of x.

```

Add the following to the end of [vector.special]:

```

template <class H, class T, class Allocator>
H hash_value(H h, const vector<T, Allocator>& x);

Requires: H meets the requirements of HashCode.
Remarks: This function shall not participate in overload resolution unless T is Hashable by H.
Returns: An object of type H whose internal state consists of the internal state of h, followed by a hash

```

representation (17.6.3.X) of x.

Associative containers

Revise the synopsis of <map> in [associative.map.syn] as follows:

```
...
template <class Key, class T, class Compare, class Allocator>
void swap(map<Key, T, Compare, Allocator>&
          map<Key, T, Compare, Allocator>&
          noexcept(noexcept(x.swap(y))));
template<class H, class Key, class T, class Compare, class Allocator>
H hash_value(H h, const map<Key, T, Compare, Allocator>& x);

...
template <class Key, class T, class Compare, class Allocator>
void swap(multimap<Key, T, Compare, Allocator>& x,
          multimap<Key, T, Compare, Allocator>& y)
    noexcept(noexcept(x.swap(y)));
template<class H, class Key, class T, class Compare, class Allocator>
H hash_value(H h, const multimap<Key, T, Compare, Allocator>& x);
}
```

Revise the synopsis of <set> in [associative.set.syn] as follows:

```
...
template <class Key, class Compare, class Allocator>
void swap(set<Key, Compare, Allocator>& x
          set<Key, Compare, Allocator>& y)
    noexcept(noexcept(x.swap(y)));
template<class H, class Key, class Compare, class Allocator>
H hash_value(H h, const set<Key, Compare, Allocator>& x);

...
template <class Key, class Compare, class Allocator>
void swap(multiset<Key, Compare, Allocator>& x,
          multiset<Key, Compare, Allocator>& y)
    noexcept(noexcept(x.swap(y)));
template<class H, class Key, class Compare, class Allocator>
H hash_value(H h, const multiset<Key, Compare, Allocator>& x);
}
```

Add the following to the end of [map.special]:

```
template<class H, class Key, class T, class Compare, class Allocator>
H hash_value(H h, const map<Key, T, Compare, Allocator>& x);

Requires: H meets the requirements of HashCode.
Remarks: This function shall not participate in overload resolution unless T is Hashable by H.
Returns: An object of type H whose internal state consists of the internal state of h, followed by a hash
representation (17.6.3.X) of x.
```

Add the following to the end of [multimap.special]:

```
template<class H, class Key, class T, class Compare, class Allocator>
H hash_value(H h, const multimap<Key, T, Compare, Allocator>& x);

Requires: H meets the requirements of HashCode.
Remarks: This function shall not participate in overload resolution unless T is Hashable by H.
Returns: An object of type H whose internal state consists of the internal state of h, followed by a hash
representation (17.6.3.X) of x.
```

Add the following to the end of [set.special]:

```
template<class H, class Key, class Compare, class Allocator>
H hash_value(H h, const set<Key, Compare, Allocator>& x);
```

Requires: H meets the requirements of `HashCode`.

Remarks: This function shall not participate in overload resolution unless T is `Hashable` by H .

Returns: An object of type H whose internal state consists of the internal state of h , followed by a hash representation (17.6.3.X) of x .

Add the following to the end of `[multiset.special]`:

```
template<class H, class Key, class Compare, class Allocator>
H hash_value(H h, const multiset<Key, Compare, Allocator>& x);
```

Requires: H meets the requirements of `HashCode`.

Remarks: This function shall not participate in overload resolution unless T is `Hashable` by H .

Returns: An object of type H whose internal state consists of the internal state of h , followed by a hash representation (17.6.3.X) of x .

`thread::id`

Revise the synopsis of `<thread::id>` in `[thread.thread.id]` as follows:

```
...
// Hash support
template <class T> struct hash;
template <> struct hash<thread::id>;

template <class H> H hash_value(H h, thread::id x);
}
```

Add the following to the end of `[thread.thread.id]`:

```
template <class H> H hash_value(H h, thread::id x);
```

Requires: H meets the requirements of `HashCode`.

Returns: An object of type H whose internal state consists of the internal state of h , followed by a hash representation (17.6.3.X) of x .

[1] The trailing length ensures that the hash representation is suffix-free. This explains why we require it to be suffix-free rather than prefix-free: to make it prefix-free, we would have to put the length before the list of elements, and that would require two traversals for containers that don't store an explicit length.

[2] `std::array` does face a difficult choice, though: heretofore it has managed to be both a sequence container and a tuple-like type, but this framework forces it to choose a side. In other words, `std::array<int, 3>` can have the same hash representation as `std::tuple<int, int, int>`, or it can have the same hash representation as `std::vector<int>`, but not both (unless we want to tack an otherwise-useless `size_t` onto the hash representations of `tuple` and `pair`).

[3] Consequently, this optimization would benefit from the availability of contiguous iterator support, as proposed by [N4183](#). Failing that, it would work only for pointer ranges.

[4] But note that it may not actually invoke `hash_value()`: if the value is uniquely represented, it may hash the bytes directly, without ever invoking user code.